Proc objects have essentially the same syntax as blocks, except that they start with ^ rather than having braces:

^*parameters* ‹[self: *self_type*]› ‹block› -> *return_type*

There are a couple of other things you can do with inside an RBS class or module definition:

- You can include, extend, or prepend modules just as you can in Ruby, and with the same meaning.
- The private, protected, and public modifiers can be used on methods or attributes and specify the access that the member will have in the actual class. They can either be directly modifying individual methods or used on their own, as they can be in Ruby.
- You can use alias to declare that two methods are the same.

RBS Types

To this point, we've kind of hand-waved what goes into the place where RBS asks for a "type." A type can be several things in RBS.

A type can be a literal value, in which case the object of this type can only have that one literal value. This seems of little practical use, but you can do it.

A type can be a class, model, or interface name (or a declared alias to one of those things), which declares that the member must be an instance of that class or a subclass of that class:

```
def height: () -> Integer
```

If you want to specify that your type is the singleton class, then you use singleton(Type), so singleton(User), for example.

In some cases, the type is a container of other objects, and you can specify the type of the objects contained using square brackets, so Array[String] or Hash[Symbol, Integer]. We'll see how to implement this in RBS Generics, on page 490.

You can specify Hash objects with fixed keys with key: type syntax, as in {name: String, height: Integer} and you can specify Array types with fixed lengths with Array syntax, as in [Integer, String, Integer].

By default, types declared with RBS can not have nil values. To specify that nil is a potential value, you append the type with a ?, as in String?. This is called an *optional type*. Note that if you have an optional parameter that is also an optional type, you can have both leading and trailing question marks: ?String?.

You can combine types. The pipe character indicates a *union*, meaning that the value can be a member of one or more of the grouped types. A common use case is for a method to take either a record or the ID of that record, which you could annotate as (id_or_record: User | Integer). Somewhat more rarely, the & indicates an intersection, meaning the value has to match all the types in the group, which you'd most commonly see as a set of interfaces: (admin: Customer & Employee).

RBS defines a handful of keywords that stand in for various kinds of types, often in the context of the class or method being defined:

- bool: an alias of true | false.
- boolish: an alias of top, which means you are expecting an arbitrary value here but treating it as a boolean. It is recommended over bool for method arguments and return values.